

Assignment A2 Part 2

Task 3

Array Sum Problem (Size = 100,000,000)					
	Sequential	1 Thread	2 Thread	3 Thread	4 Thread
Time	real 0m0.355s user 0m0.096s sys 0m0.259s	real 0m0.646s user 0m0.104s sys 0m0.540s	real 0m0.544s user 0m0.123s sys 0m0.448s	real 0m0.462s user 0m0.102s sys 0m0.402s	real 0m0.445s user 0m0.126s sys 0m0.370s

Matrix Multiplication Problem (Size = 1,000)					
	Sequential	1 Thread	2 Thread	3 Thread	4 Thread
Time	real 0m1.336s user 0m1.332s sys 0m0.004s	real 0m1.233s user 0m1.226s sys 0m0.004s	real 0m0.591s user 0m1.152s sys 0m0.008s	real 0m0.458s user 0m1.337s sys 0m0.008s	real 0m0.368s user 0m1.385s sys 0m0.008s

Task 4

For the array sum problem, my multithreaded implementation had comparable, or even worse performance than the sequential implementation. The matrix multiplication problem was the program that clearly benefitted from a multi-threaded implementation.

I didn't use synchronization primitives because each thread had its own dedicated slice of memory to work with. Therefore, there was no chance of a race condition occurring. Although, I did find the idea of using a mutex simple and pleasant, especially compared to forking processes and having to pipe data between parent and child processes.

Regarding the code snippet, although I would like to think that lock variable would suffice, my gut tells me that a race condition may occur. I think if two or more threads execute in close enough proximity, they may both evaluate the lock variable as either a 0 or a 1.